

Phase 3 Code Challenge: Articles - without SQLAlchemy (Updated) (CodeGrade)

 <https://github.com/learn-co-curriculum/python-non-sqlalchemy-code-challenge-articles>  <https://github.com/learn-co-curriculum/python-non-sqlalchemy-code-challenge-articles/issues/new>

For this assignment, you will be working with a Magazine domain.

We have three models: `Author` , `Article` , and `Magazine` .

For our purposes, an `Author` has many `Article` s, a `Magazine` has many `Article` s, and `Article` s belong to both `Author` and `Magazine` .

`Author` - `Magazine` is a many to many relationship.

Note: You should draw your domain on paper or on a whiteboard *before you start coding*. Remember to identify a single source of truth for your data.

Topics

- Classes and Instances
- Class and Instance Methods
- Variable Scope
- Object Relationships
- lists and list Methods

Instructions

To get started, run `pipenv install` while inside of this directory. Then run `pipenv shell` to jump into the shell.

Build out all of the methods listed in the deliverables. The methods are listed in a suggested order, but you can feel free to tackle the ones you think are easiest. Be careful: some of the later methods rely on earlier ones.

Remember! This code challenge has tests to help you check your work. You can run `pytest` to make sure your code is functional before submitting.

We've provided you with a tool that you can use to test your code. To use it, run `python lib/debug.py` from the command line. This will start a `ipdb` session with your classes defined. You can test out the methods that you write here. You can add code to the `lib/debug.py` file to define variables and create sample instances of your objects.

Writing error-free code is more important than completing all of the deliverables listed - prioritize writing methods that work over writing more methods that don't work. You should test your code in the console as you write.

Similarly, messy code that works is better than clean code that doesn't. First, prioritize getting things working. Then, if there is time at the end, refactor your code to adhere to best practices. When you encounter duplicated logic, extract it into a shared helper method.

Before you submit! Save and run your code to verify that it works as you expect. If you have any methods that are not working yet, feel free to leave comments describing your progress.

Deliverables

Write the following methods in the classes in the files provided. Feel free to build out any helper methods if needed.

Initializers and Properties

Author

- `Author __init__(self, name)`
 - Author is initialized with a name
- `Author property name`
 - Returns the author's name
 - Names must be of type `str`
 - Names must be longer than 0 characters

- Should **not be able** to change after the author is instantiated.
- *hint: hasattr()*

Magazine

- `Magazine __init__(self, name, category)`
 - A magazine is initialized with a name and a category
- `Magazine property name`
 - Returns the magazine's name
 - Names must be of type `str`
 - Names must be between 2 and 16 characters, inclusive
 - Should **be able** to change after the magazine is instantiated.
- `Magazine property category`
 - Returns the magazine's category
 - Categories must be of type `str`
 - Categories must be longer than 0 characters
 - Should **be able** to change after the magazine is instantiated.

Article

- `Article __init__(self, author, magazine, title)`
 - Article is initialized with an `Author` instance, a `Magazine` instance, and a title.
- `Article property title`
 - Returns the article's title
 - Titles must be of type `str`
 - Titles must be between 5 and 50 characters, inclusive
 - Should **not be able** to change after the article is instantiated.
 - *hint: hasattr()*

Object Relationship Methods and Properties

Article

- **Article property author**
 - Returns the author object for that article
 - Must be of type **Author**
 - Authors **can be changed** after the article object is initialized
- **Article property magazine**
 - Returns the magazine object for that article
 - Must be of type **Magazine**
 - Magazines **can be changed** after the article object is initialized

Author

- **Author articles()**
 - Returns a list of all the articles the author has written
 - Must be of type **Article**
- **Author magazines()**
 - Returns a **unique** list of magazines for which the author has contributed to
 - Must be of type **Magazine**

Magazine

- **Magazine articles()**
 - Returns a list of all the articles the magazine has published
 - Must be of type **Article**
- **Magazine contributors()**
 - Returns a **unique** list of authors who have written for this magazine
 - Must be of type **Author**

Aggregate and Association Methods

Author

- **Author add_article(magazine, title)**
 - Receives a **Magazine** instance, and a title as arguments

- Creates and returns a new **Article** instance and associates it with that author, the magazine provided
- **Author topic_areas()**
 - Returns a **unique** list of strings with the categories of the magazines the author has contributed to
 - Returns **None** if the author has no articles

Magazine

- **Magazine article_titles()**
 - Returns a list of the titles strings of all articles written for that magazine
 - Returns **None** if the magazine has no articles
- **Magazine contributing_authors()**
 - Returns a list of authors who have written more than 2 articles for the magazine
 - Authors must be of type **Author**
 - Returns **None** if the magazine has no authors with more than 2 publications

Bonus: Aggregate and Association Method

- **Magazine classmethod top_publisher()**
 - Returns the **Magazine** instance with the most articles
 - Returns **None** if there are no articles.
 - Uncomment lines 206-224 in the magazine_test file
 - *hint: will need a way to remember all magazine objects*

Bonus: For any invalid inputs raise an **Exception** .

- First, **comment out** the following lines
 - **article_test.py**
 - lines 28-29
 - **author_test.py**
 - lines 31-32, and 35-36
 - **magazine_test.py**
 - lines 31-32, 47-48, 51-52, 84-85, and 100-102

- Then, **uncomment** the following lines in the test files
 - **article_test.py**
 - lines 34-35, 46-47, and 50-51
 - **author_test.py**
 - lines 39-40, and 53-54
 - **magazine_test.py**
 - lines 35-36, 55-56, 59-60, 90-91, and 105-106

This tool needs to be loaded in a new browser window

Load Phase 3 Code Challenge: Articles - without SQLAlchemy (Updated) (CodeGrade) in a new window